]

```
    for x, y, z in winning_combinations: if board[x] == board[y] == board[z] and board[x] != " ":
return board[x]
    return None

def is_draw(board):
    return " " not in board and check_winner(board) is None # Q-Table for states and actions q_table
= {}

# Function to convert board to a string for Q-Table keys def board_to_string(board):
    return "".join(board)

# Training the AI
learning_rate = 0.1
reward_win = 1
reward_loss = -1
reward_draw = 0.5
gamma = 0.9

for episode in range(10000):
    board = [" "] * 9
    state = board_to_string(board)
    while " " in board:
        # AI chooses action
        if state not in q_table:
            q_table[state] = [0] * 9 # Initialize Q-values for all actions in this state available_moves =
[i for i, v in enumerate(board) if v == " "]
        move = random.choice(available_moves) board[move] = "X"
        next_state = board_to_string(board) # Check for winner or opponent's turn winner =
check_winner(board)
        if winner == "X":
            q_table[state][move] += learning_rate * (reward_win - q_table[state][move]) break
        elif is_draw(board):
            q_table[state][move] += learning_rate * (reward_draw - q_table[state][move]) break

        # Opponent plays randomly
        opponent_move = random.choice([i for i, v in enumerate(board) if v == " "])
board[opponent_move] = "O"
        winner = check_winner(board)

        if winner == "O":
            q_table[state][move] += learning_rate * (reward_loss - q_table[state][move]) break
        elif is_draw(board):
            q_table[state][move] += learning_rate * (reward_draw - q_table[state][move]) break

        # Update Q-value
        next_state = board_to_string(board) if next_state not in q_table:
            q_table[next_state] = [0] * 9
        q_table[state][move] += learning_rate * (
```